

Übung SW15: Abschlussübung und Repetition

Themen: Code-Review, statische Codeanalyse, Designprinzipien (P03).
Zeitbedarf: ca. 180min.

Roland Gisler, Version 1.2.7 (FS26)

1 Statische Codeanalyse aller Übungen

1.1 Lernziele

- Vorhandenes Buildsystem zu Codeanalyse nutzen.
- QS-Reports (Checkstyle, PMD und SpotBugs) studieren.
- Code aller Übungsaufgaben reviewen und verbessern.

1.2 Grundlagen

In den vergangenen vierzehn Semesterwochen haben Sie sehr viel gelernt und einiges an Code geschrieben. Diesen Code wollen wir uns zum Abschluss des Semesters nochmal ansehen. Es lohnt sich!

1.3 Aufgaben

- a.) Vorbereitung: Sorgen Sie dafür, dass Ihr Projekt fehlerfrei kompilierbar ist und in der IDE keine Fehler anzeigt. Am einfachsten prüfen Sie das mit `mvn clean package` (direkt in einer Shell) welches mit `BUILD SUCCESS` abschliesst.
- b.) Das Maven-Template, das Sie seit SW05 verwenden, kann noch viel mehr als Sie bisher genutzt haben! Starten Sie mal einen Build mit `mvn site`! Bei einem erfolgreichen Abschluss finden Sie danach das Verzeichnis `./target/site`, und darin ein `index.html`. Öffnen Sie die Datei und staunen Sie!

Hinweis: Einer der häufigsten Fehler, wenn «`mvn site`» nicht erfolgreich abgeschlossen werden kann ist, dass Sie Fehler in der JavaDoc haben. Es ist mühsam, aber lohnend diese Fehler (mindestens die «errors», die «warnings» sind optional) Stück für Stück zu korrigieren! Zur Not können Sie die fehlerhafte JavaDoc auch löschen. Denn eine «fehlende» JavaDoc ist zweifelsfrei immer noch «besser» als eine fehlerhafte JavaDoc!

- c.) Die interessantesten Informationen sind unter «Project Reports» zu finden: «**Checkstyle**», «**PMD**» und «**SpotBugs**». Lassen Sie sich von der (vermutlich grossen) Menge an Meldungen nicht abschrecken! Viele der Meldungen können Sie sogar mit einer automatischen Codeformatierung und/oder dem Reorganisieren der Imports in der IDE sehr leicht beheben. Führen Sie dann einfach nochmal «`mvn site`» aus, und sehen Sie dann die Verbesserung!

Hinweis: Checkstyle meldet die «einfachsten» Fehler und Beanstandungen. PMD ist etwas «intelligenter». SpotBugs geht am tiefsten. Beachten Sie, dass es immer auch «false positives» geben kann, also Beanstandungen, die Sie selbst anders beurteilen. Das ist auch erlaubt, ohne spezifische Konfiguration erreichen Projekte selten null Beanstandungen!

- d.) Studieren Sie die Liste der Befunde, und suchen Sie interessante (oder unverständliche) Befunde. Diskutieren Sie diese mit anderen Studierenden oder den Dozierenden!

2 Verbessern der Codeabdeckung und der Testfälle

2.1 Lernziele

- Nicht oder schlecht getesteten Code identifizieren.
- Schreiben von Unit Tests, optional mit AssertJ.
- Professionelles Vorgehen beim Entwickeln von Software.

2.2 Grundlagen

Quellcode und insbesondere bereits vorhandene Unittests zu allen bisherigen Übungen (bzw. seit der Einführung von Unit Testing).

2.3 Aufgaben

- a.) Wenn Sie die Aufgabe 1 gelöst haben, finden Sie in der erstellten Website unter dem Titel «**JaCoCo**» auch einen Bericht zur Codeabdeckung. Diesem Report können Sie entnehmen, welche Codezeilen von den Testfällen wirklich ausgeführt und somit getestet wurden. Studieren Sie den Report! Sind Sie zufrieden mit dem erreichten Wert?

Hinweis: Auch wenn das höchste Ziel natürlich bei 100% liegt, ist es in vielen Projekten nicht realistisch und/oder nicht ökonomisch sinnvoll zu erreichen. Aber mehr geht immer! 😊

- b.) Identifizieren Sie Codeabschnitte, bei welchen Sie erkennen, dass Sie mit wenig Aufwand (z.B. einem zusätzlichen Testfall) die Codeabdeckung sehr leicht erhöhen können. Ergänzen Sie diese Testfälle! Das ist eine lohnenswerte Übung zum Schreiben von Unit Tests, zumal wir das Thema bei den Übungen zu I/O-Streams und GUI's (beides ist schwierig mit Unit Tests zu prüfen) potentiell ein bisschen vernachlässigt haben. Konzentrieren Sie sich ruhig auf die «einfachen» Fälle!

Hinweis: Schreiben Sie dafür unbedingt **neue** Unit Tests und vermeiden Sie es, bereits bestehende Unit Tests zu erweitern. Wissen Sie noch warum?

- c.) Versuchen Sie doch mal mit AssertJ zu arbeiten! Diese Library erlaubt es Ihnen, **assert**-Statements noch wesentlich ausdrückstärker und mit einer Fluent-API zu schreiben. Die Library ist bereits im Projekt integriert. Sie finden unter <https://assertj.github.io/doc/> eine ausführliche Dokumentation mit vielen Beispielen!

3 Repetitionsaufgaben der letzten Wochen

3.1 Lernziele

- Repetition bereits behandelter Themen zur Festigung und Vertiefung.

3.2 Grundlagen

Diese Zusatzaufgaben sind unabhängig von den restlichen Aufgaben dieser Woche. Sie baut aber auf der optionalen Repetitionsaufgabe der letzten Woche auf.

3.3 Aufgaben

- a.) Haben Sie die Repetitionsaufgaben der letzten fünf Wochen auch gelöst?

Wenn ja: **Herzliche Gratulation!**

Diese Aufgaben sind nämlich eine gute Vorbereitung auf die Modulendprüfung, da sie aus einer vergangenen Modulendprüfung (FS22) stammen. Für jede Aufgabe wurden damals 18 Punkte vergeben - jede Aufgabe entspricht somit einem ungefähren Arbeitsaufwand von 20min.

Wenn nein: Vielleicht haben Sie jetzt die Motivation, diese Aufgaben noch zu lösen? 😊

- b.) **Blicken Sie mit Stolz auf die vergangenen vierzehn Wochen Programmieren zurück!**
Sie haben in dieser Zeit wirklich sehr viel gelernt!